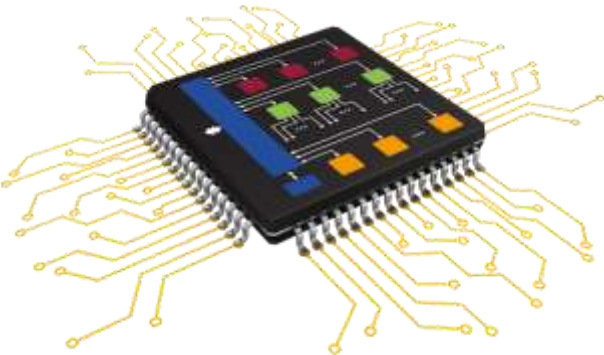


# **MODULE - 3**

## **ARCHITECTURE OF SPECIAL PURPOSE COMPUTING SYSTEM**



# MODULE - 3

## Architecture of Special Purpose Computing System

ATM, Handheld devices, Data Compressor, Image Capturing Devices Architecture and Requirements, Challenges Constraints of special purpose computing system.

# Data Compression

# Why Data Compression?

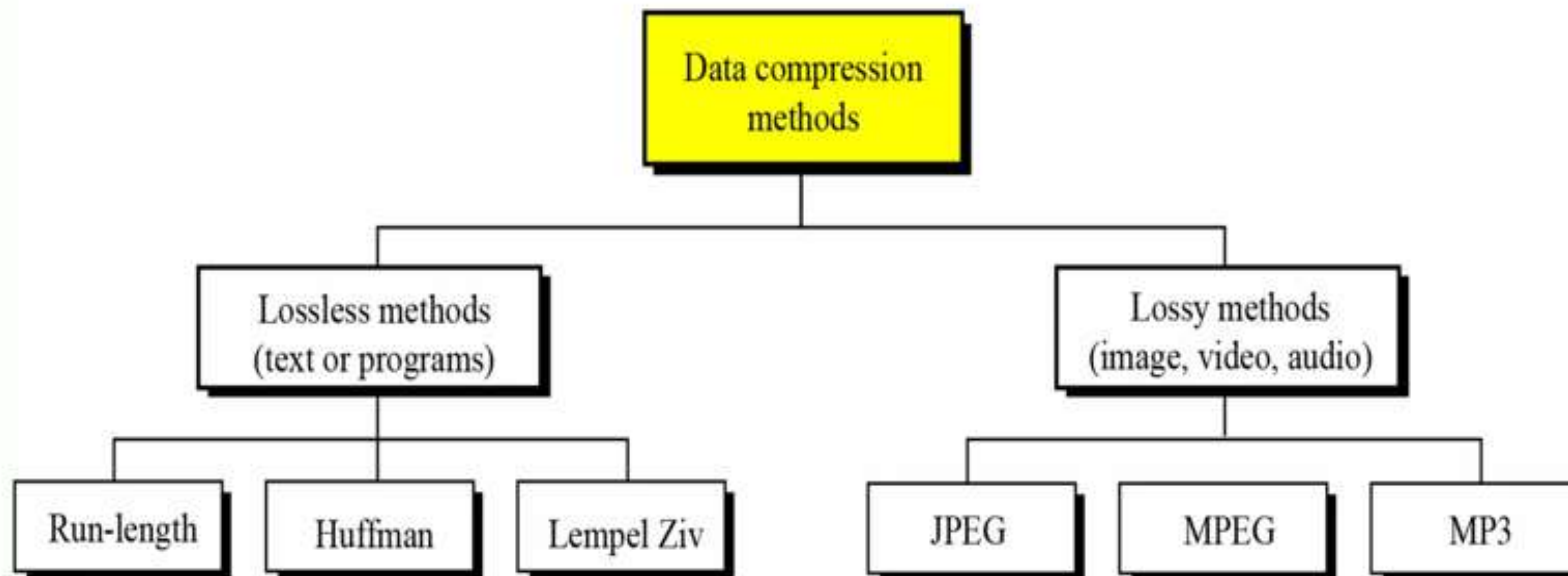
- Make optimal use of limited storage space
- Save time and help to optimize resources
  - If compression and decompression are done in I/O processor, less time is required to move data to or from storage subsystem, freeing I/O bus for other work
  - In sending data over communication line: less time to transmit and less storage to host

# Data Compression- Entropy

- Entropy is the measure of information content in a message.
  - Messages with higher entropy carry more information than messages with lower entropy.
- How to determine the entropy
  - Find the probability  $p(x)$  of symbol  $x$  in the message
  - The entropy  $H(x)$  of the symbol  $x$  is:
$$H(x) = - p(x) \cdot \log_2 p(x)$$
- The average entropy over the entire message is the sum of the entropy of all  $n$  symbols in the message

# Data Compression Methods

- Data compression is about storing and sending a smaller number of bits.
- There're two major categories for methods to compress data: lossless and lossy methods





# Lossless Compression Methods

- In lossless methods, original data and the data after compression and decompression are exactly the same.
- Redundant data is removed in compression and added during decompression.
- Lossless methods are used when we can't afford to lose any data: legal and medical documents, computer programs.

# Run-length encoding

- Simplest method of compression.
- How: replace consecutive repeating occurrences of a symbol by 1 occurrence of the symbol itself, then followed by the number of occurrences.

a. Original data

BBBBBBBBBAAAAAAAAAAAAAAAAANMMMMMMMMMM

b. Compressed data

B09A16N01M10

- The method can be more efficient if the data uses only 2 symbols (0s and 1s) in bit patterns and 1 symbol is more frequent than another.

a. Original data

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0

14

4

0

12

1110 0100 0000 1100

b. Compressed data



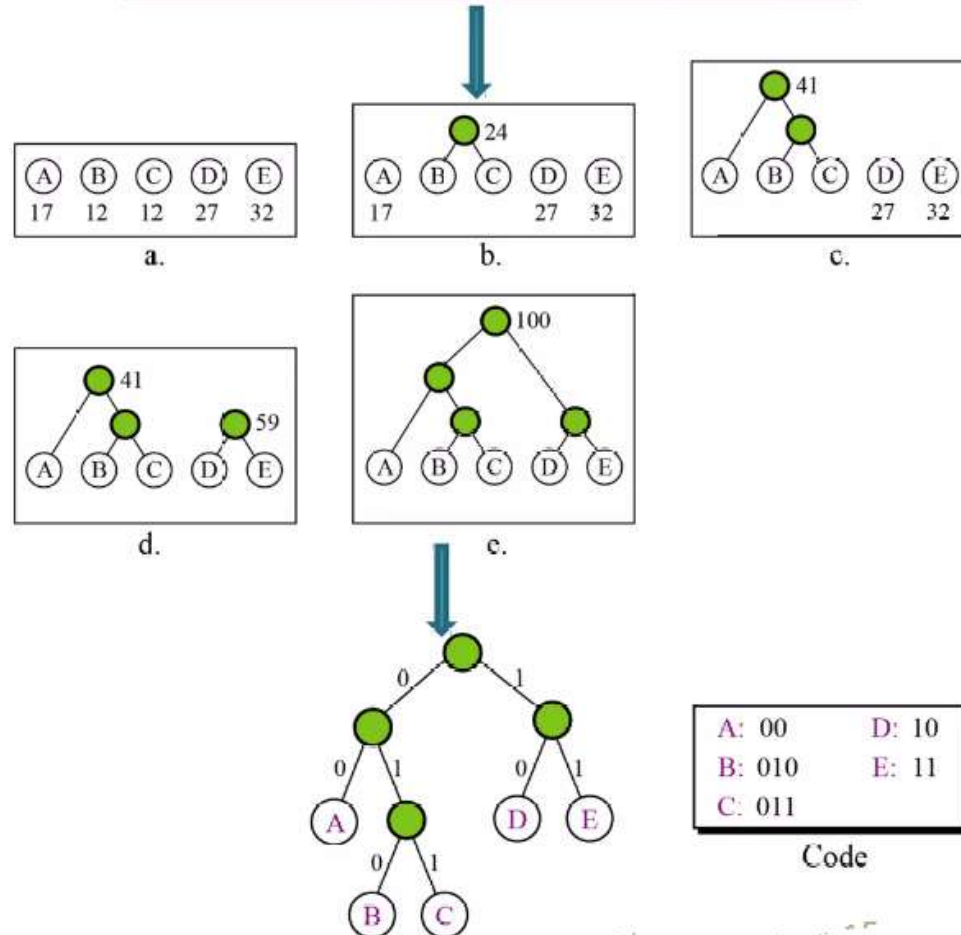
# Huffman Coding

- Assign fewer bits to symbols that occur more frequently and more bits to symbols appear less often.
- There's no unique Huffman code and every Huffman code has the same average code length.
- Algorithm:
  - a. Make a leaf node for each code symbol
    - ◆ Add the generation probability of each symbol to the leaf node
  - a. Take the two leaf nodes with the smallest probability and connect them into a new node
    - ◆ Add 1 or 0 to each of the two branches : Add 0 to the branch ending with the lesser value node and 1 to the other branch ending with greater value node
    - ◆ The probability of the new node is the sum of the probabilities of the two connecting nodes
  - a. If there is only one node left, the code construction is completed. If not, go back to (2)

● Example

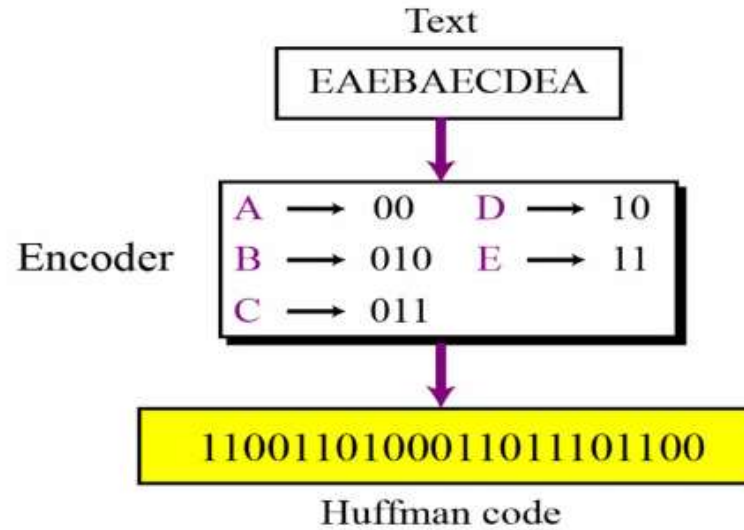
Table 15.1 Frequency of characters

| Character | A  | B  | C  | D  | E  |
|-----------|----|----|----|----|----|
| Frequency | 17 | 12 | 12 | 27 | 32 |

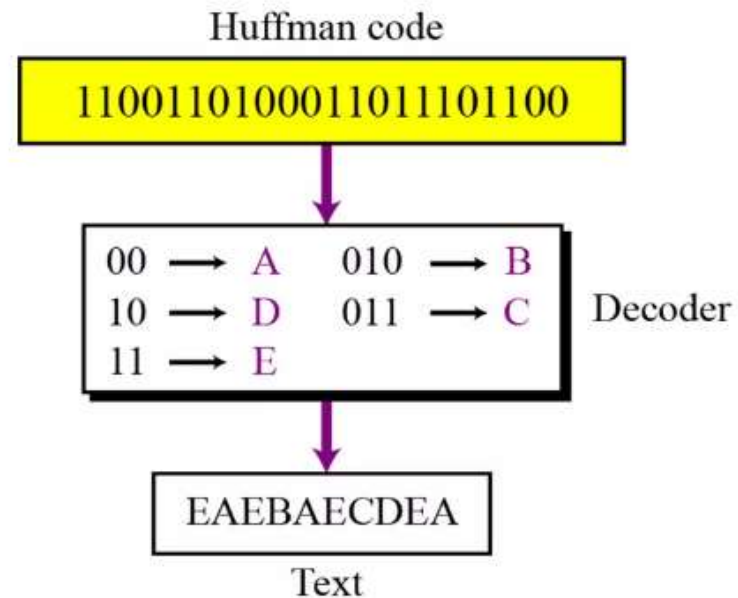


# Huffman Coding

- Encoding



- Decoding



# Lempel Ziv Encoding

- It is *dictionary-based* encoding
- Basic idea:
  - Create a dictionary(a table) of strings used during communication.
  - If both sender and receiver have a copy of the dictionary, then previously-encountered strings can be substituted by their index in the dictionary.

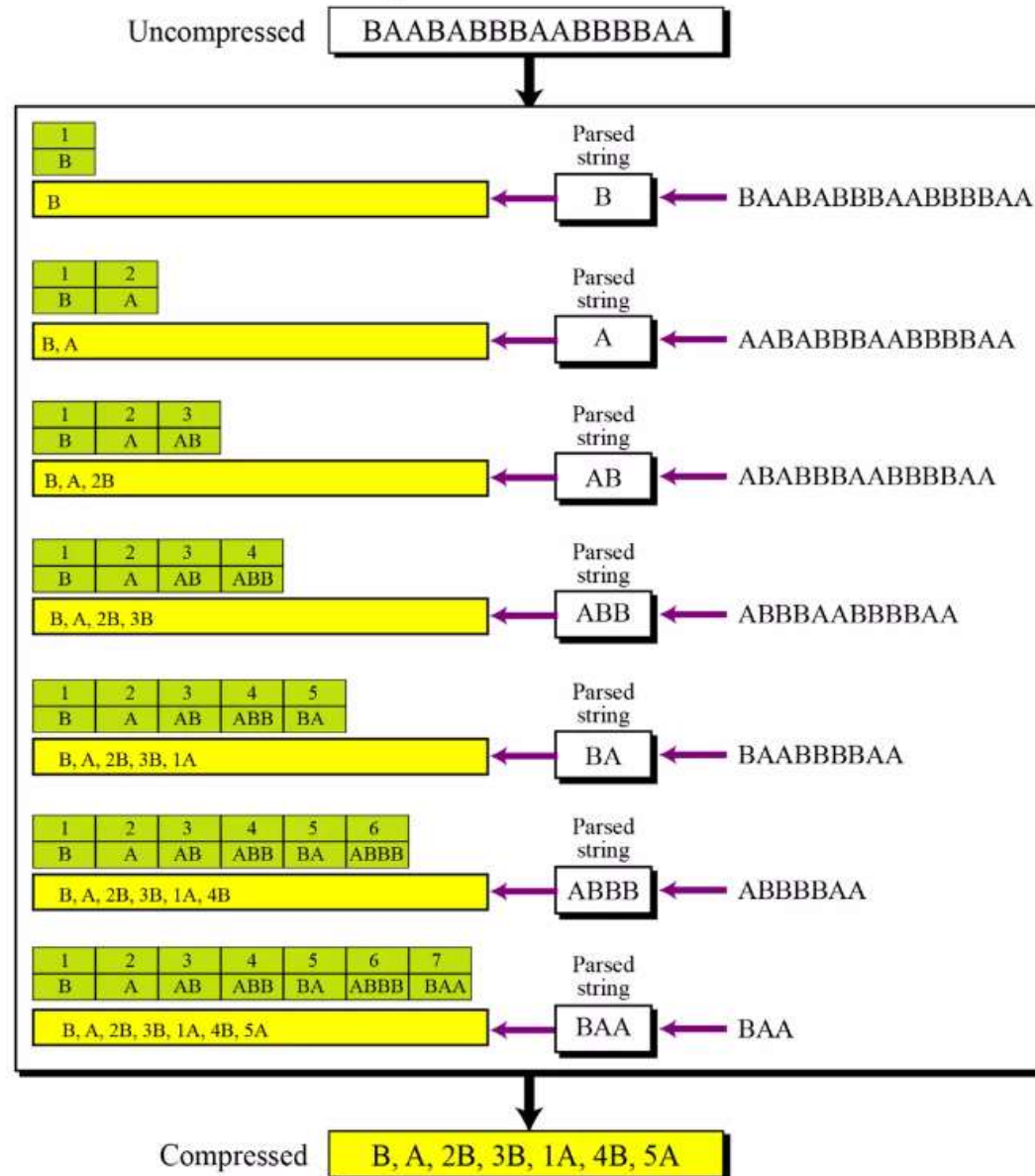
# Lempel Ziv Compression

- Have 2 phases:
  - Building an indexed dictionary
  - Compressing a string of symbols
- Algorithm:
  - Extract the smallest substring that cannot be found in the remaining uncompressed string.
  - Store that substring in the dictionary as a new entry and assign it an index value
  - Substring is replaced with the index found in the dictionary
  - Insert the index and the last character of the substring into the compressed string



# Lempel Ziv Compression

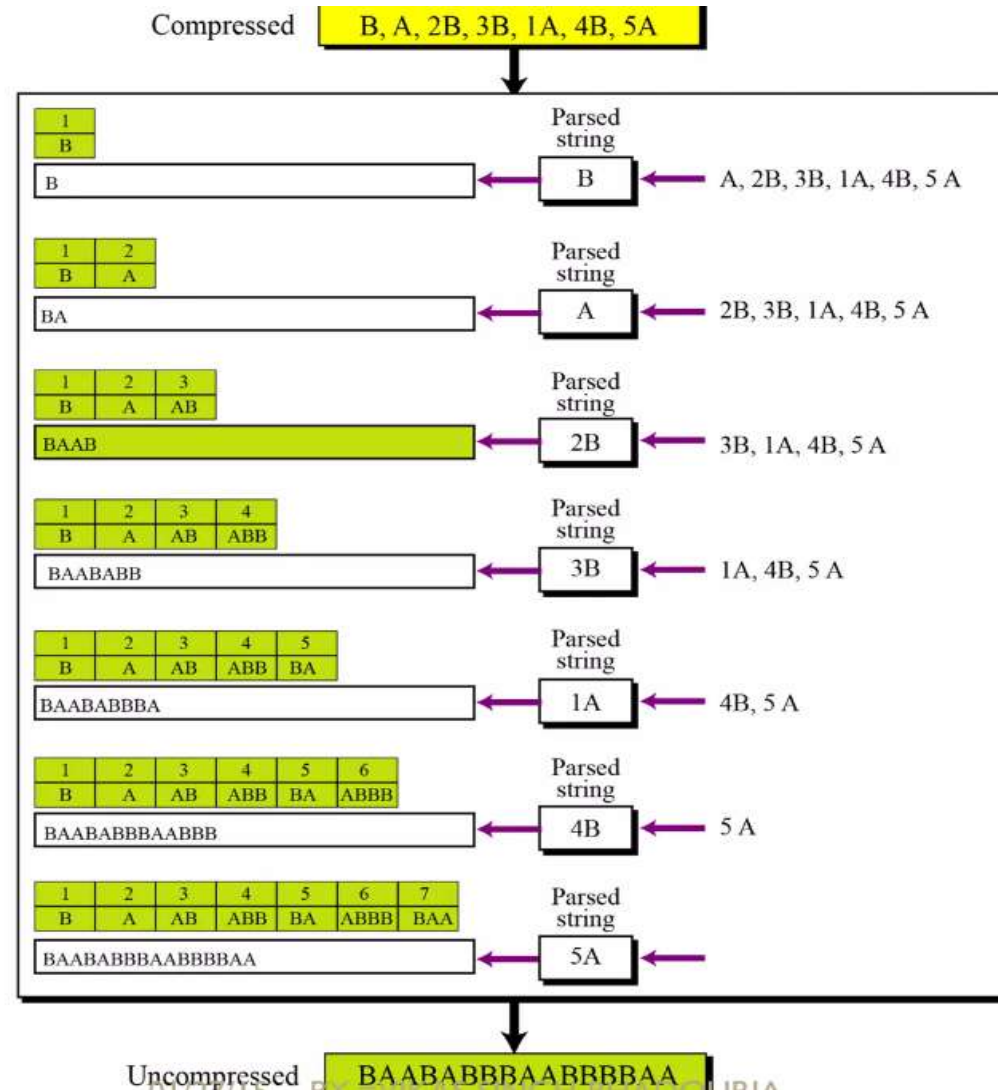
- Compression example:





# Lempel Ziv Decompression

- It's just the inverse of compression process

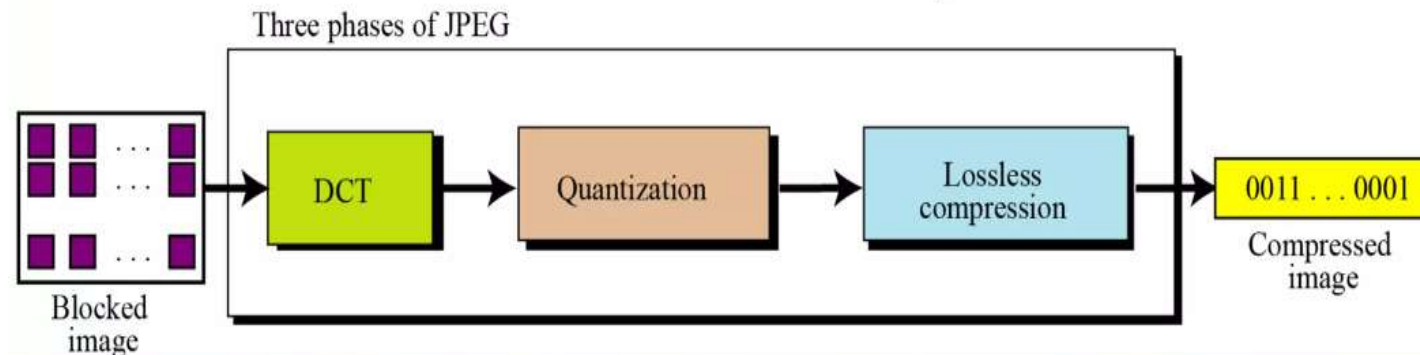


# Lossy Compression Methods

- Used for compressing images and video files (our eyes cannot distinguish subtle changes, so lossy data is acceptable).
- These methods are cheaper, less time and space.
- Several methods:
  - JPEG: compress pictures and graphics
  - MPEG: compress video
  - MP3: compress audio

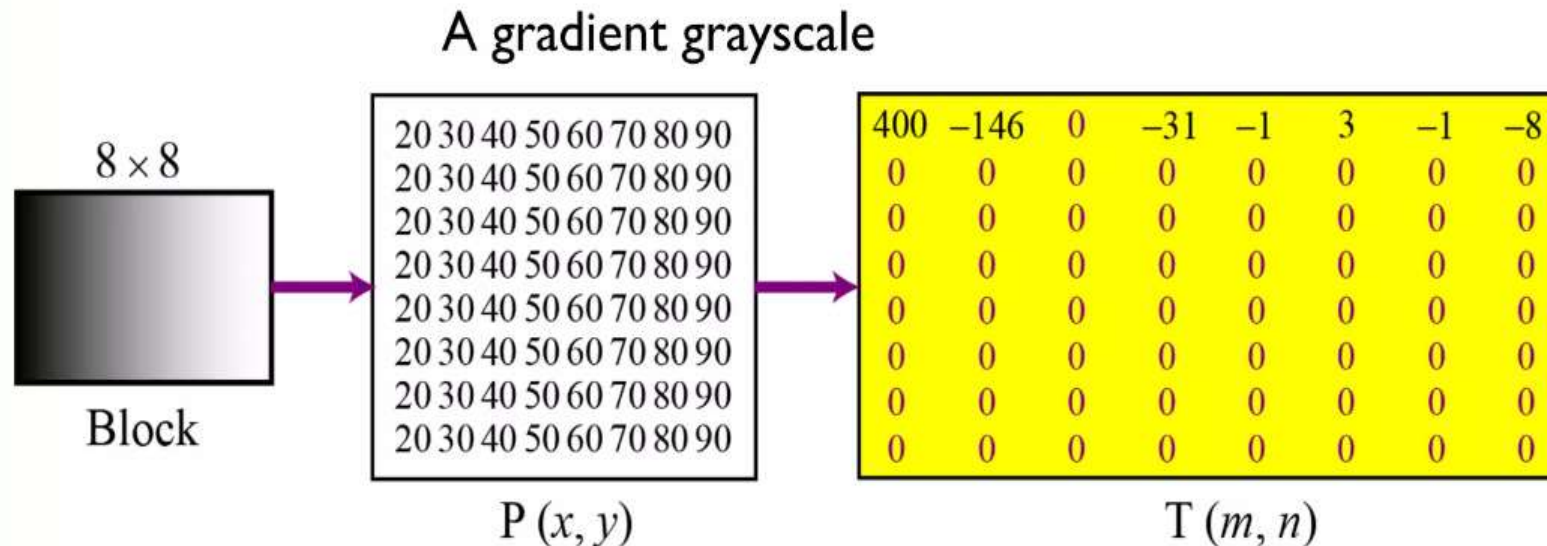
# JPEG Encoding

- Used to compress pictures and graphics.
- In JPEG, a grayscale picture is divided into 8x8 pixel blocks to decrease the number of calculations.
- Basic idea:
  - Change the picture into a linear (vector) sets of numbers that reveals the redundancies.
  - The redundancies is then removed by one of lossless



# JPEG Encoding- DCT

- DCT: Discrete Concise Transform
- DCT transforms the 64 values in 8x8 pixel block in a way that the relative relationships between pixels are kept but the redundancies are revealed.
- Example:





# Quantization & Compression

- **Quantization:**

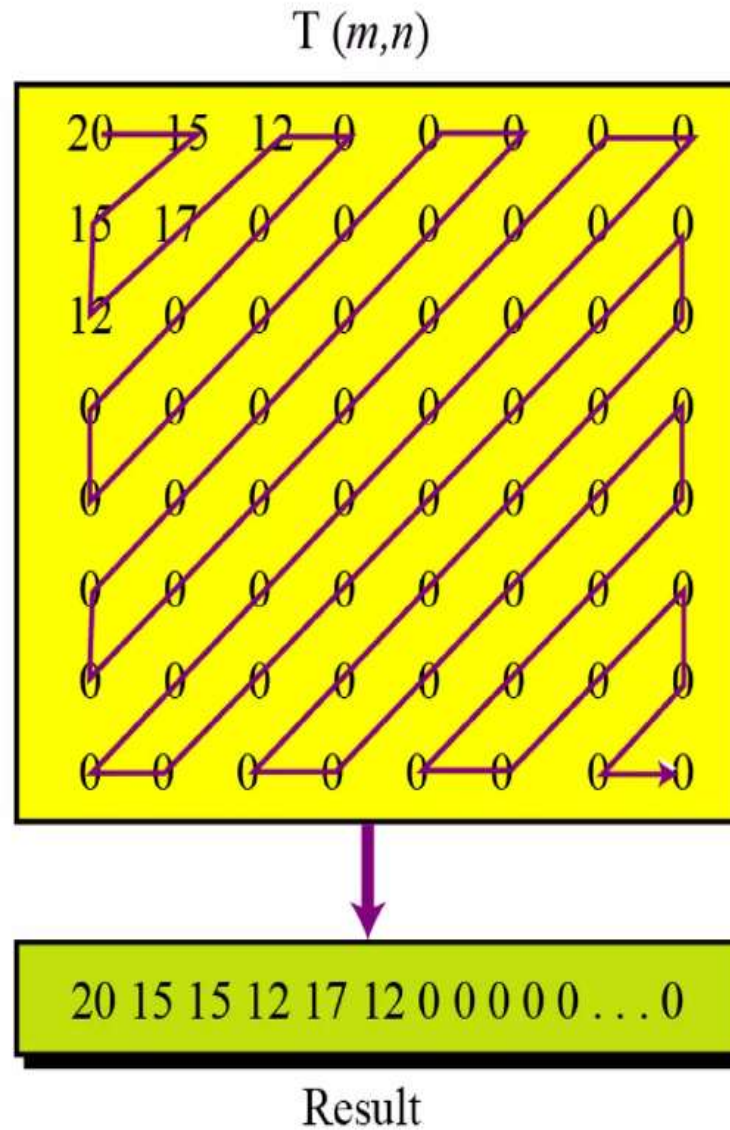
- After T table is created, the values are quantized to reduce the number of bits needed for encoding.
- Quantization divides the number of bits by a constant, then drops the fraction. This is done to optimize the number of bits and the number of 0s for each particular application.

- **Compression:**

- Quantized values are read from the table and redundant 0s are removed.
- To cluster the 0s together, the table is read diagonally in an zigzag fashion. The reason is if the table doesn't have fine changes, the bottom right corner of the table is all 0s.
- JPEG usually uses lossless run-length encoding at the compression phase.



# JPEG Encoding

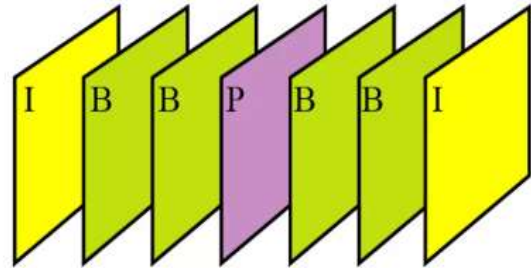


# MPEG Encoding

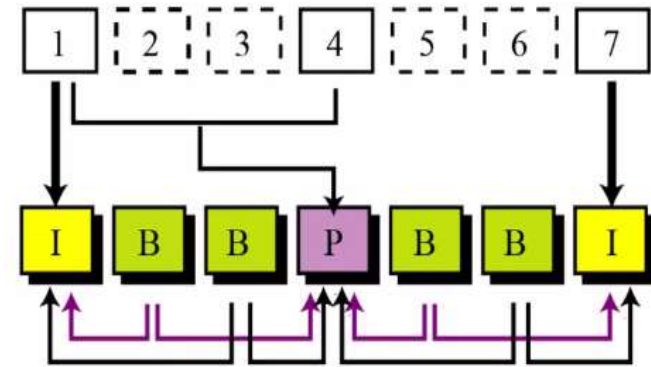
- Used to compress video.
- Basic idea:
  - Each video is a rapid sequence of a set of frames. Each frame is a spatial combination of pixels, or a picture.
  - Compressing video =  
spatially compressing each frame  
+  
temporally compressing a set of frames.

# MPEG Encoding

- Spatial Compression
  - Each frame is spatially compressed by JPEG.
- Temporal Compression
  - Redundant frames are removed.
  - For example, in a static scene in which someone is talking, most frames are the same except for the segment around the speaker's lips, which changes from one frame to the next.



a. Frames



b. Frame construction

# Audio Compression

- Used for speech or music
  - Speech: compress a 64 kHz digitized signal
  - Music: compress a 1.411 MHz signal
- Two categories of techniques:
  - Predictive encoding
  - Perceptual encoding

# Audio Encoding

- Predictive Encoding

- Only the differences between samples are encoded, not the whole sample values.
- Several standards: GSM (13 kbps), G.729 (8 kbps), and G.723.3 (6.4 or 5.3 kbps)

- Perceptual Encoding: **MP3**

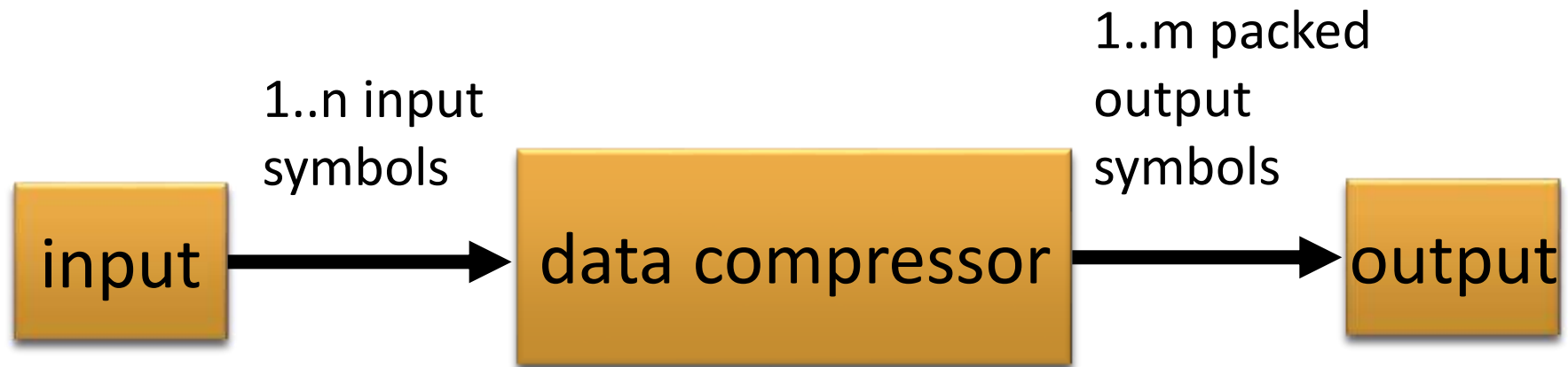
- CD-quality audio needs at least 1.411 Mbps and cannot be sent over the Internet without compression.
- MP3 (MPEG audio layer 3) uses perceptual encoding technique to compress audio.



# GOALS OF DATA COMPRESSION IN EMBEDDED SYSTEM

- ✓ Compress data transmitted over serial line.
  - ✓ Receives byte-size input symbols.
  - ✓ Produces output symbols packed into bytes.
- ✓ Will build software module only here.

# BLOCK DIAGRAM FOR COMPRESSOR



# HUFFMAN CODING

- Early statistical text compression algorithm.
- Select non-uniform size codes.
  - Use shorter codes for more common symbols.
  - Use longer codes for less common symbols.
- To allow decoding, codes must have unique prefixes.
  - No code can be a prefix of a longer valid code.

# HUFFMAN EXAMPLE

character P

a .45

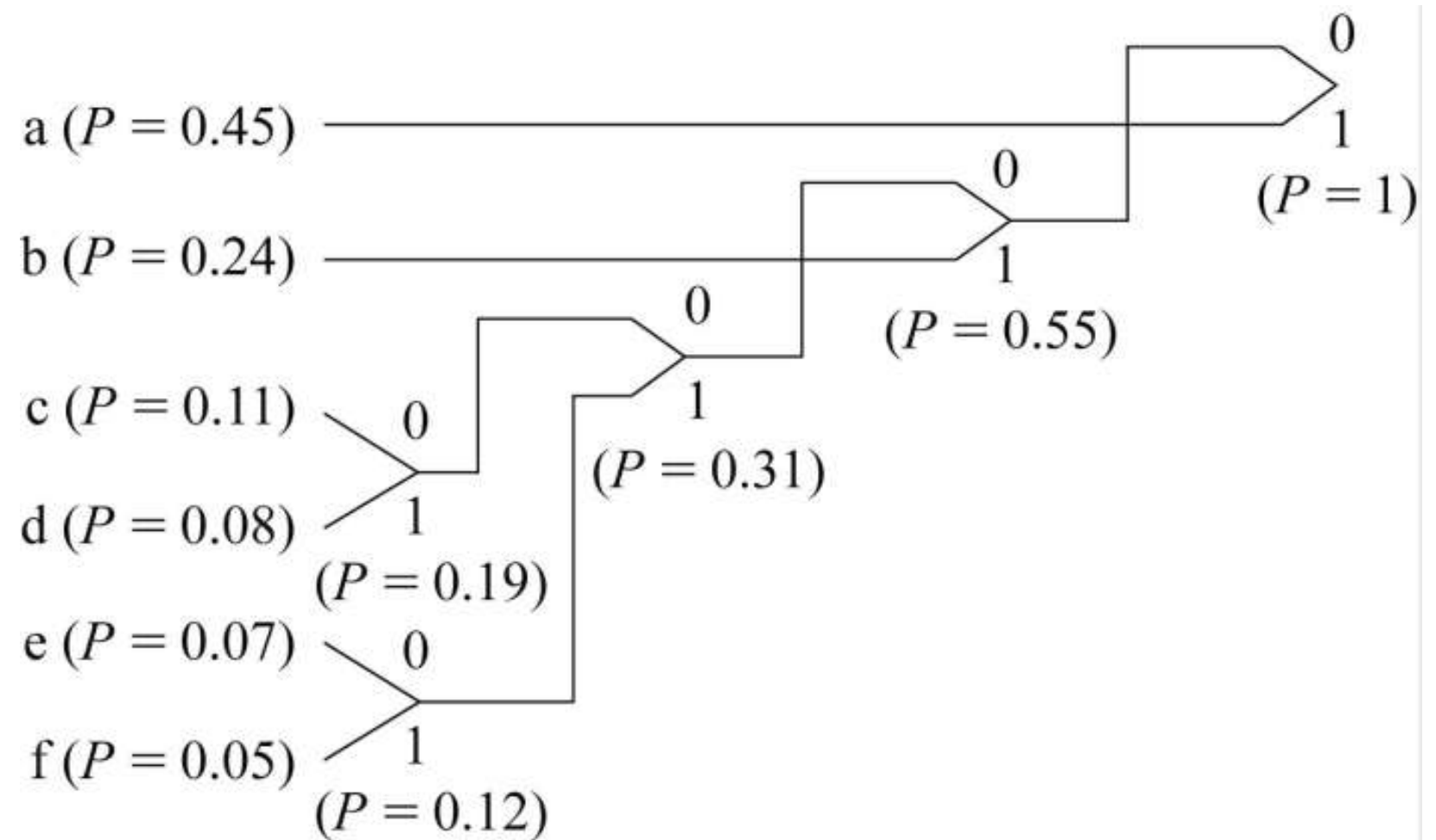
b .24

c .11

d .08

e .07

f .05



# EXAMPLE HUFFMAN CODE

Read code from root to leaves:

a    1

b    01

c    0000

d    0001

e    0010

f    0011



# REQUIREMENTS

| Name                     | Data compression module                         |
|--------------------------|-------------------------------------------------|
| Purpose                  | Code module for Huffman data compression        |
| Inputs                   | Encoding table, uncoded byte-size input symbols |
| Outputs                  | Packed compressed output symbols                |
| Functions                | Huffman coding                                  |
| Performance              | Requires fast performance                       |
| Manufacturing cost       | N/A                                             |
| Power                    | N/A                                             |
| Physical size and weight | N/A                                             |

# BUILDING A SPECIFICATION

Block diagram shows only input/output.

A real system must:

- Accept an encoding table.

- Allow a system reset that flushes the compression buffer.

# DATA-COMPRESSOR CLASS

data-compressor

buffer: data-buffer  
table: symbol-table  
current-bit: integer

encode(): boolean,  
                  data-buffer  
flush()  
new-symbol-table()

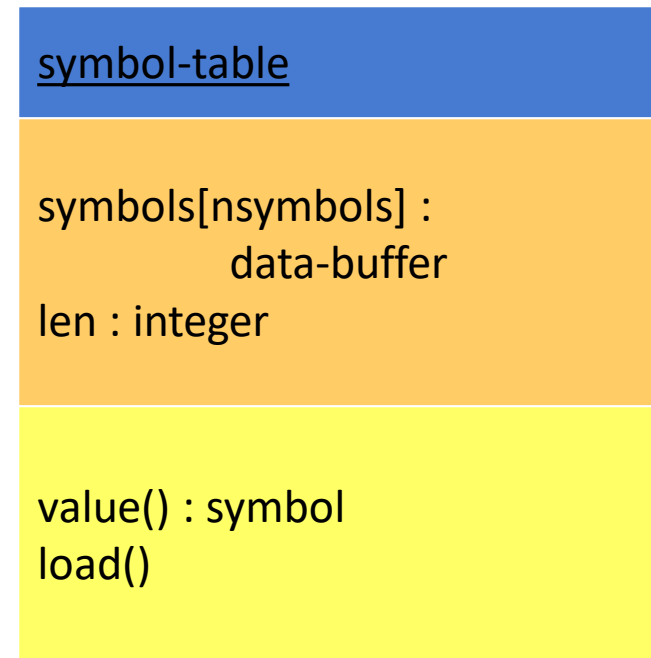
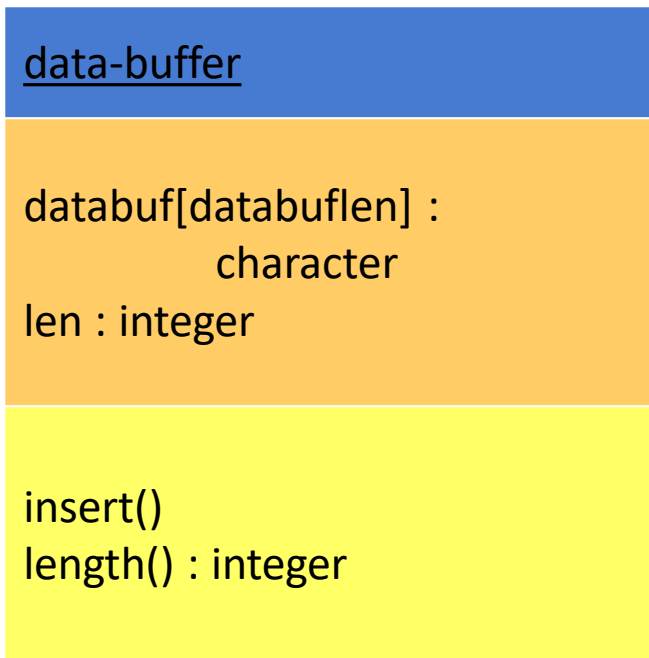
# DATA-COMPRESSOR BEHAVIORS

encode: Takes one-byte input, generates packed encoded symbols and a Boolean indicating whether the buffer is full.

new-symbol-table: installs new symbol table in object, throws away old table.

flush: returns current state of buffer, including number of valid bits in buffer.

# AUXILIARY CLASSES



# AUXILIARY CLASS ROLES

data-buffer holds both packed and unpacked symbols.

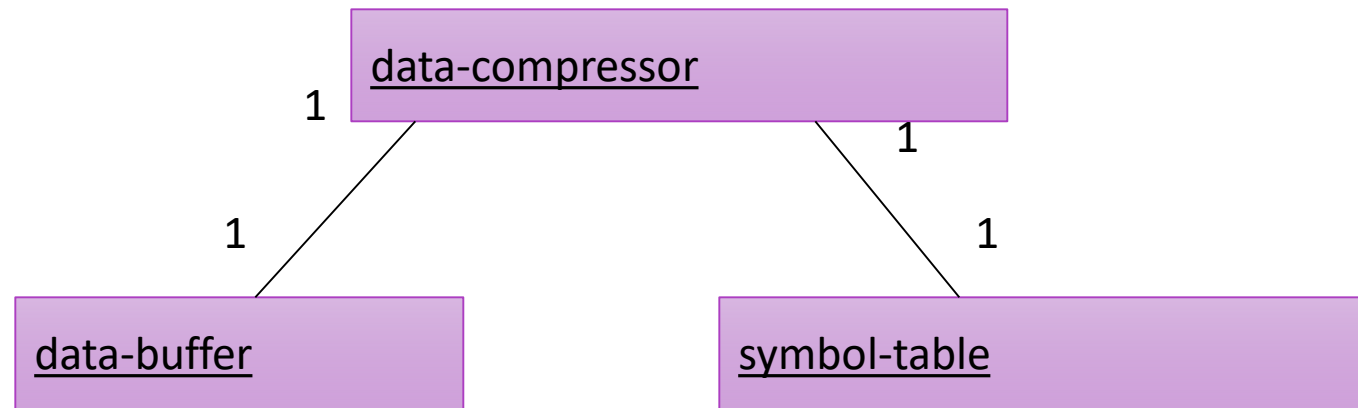
Longest Huffman code for 8-bit inputs is 256 bits.

symbol-table indexes encoded version of each symbol.

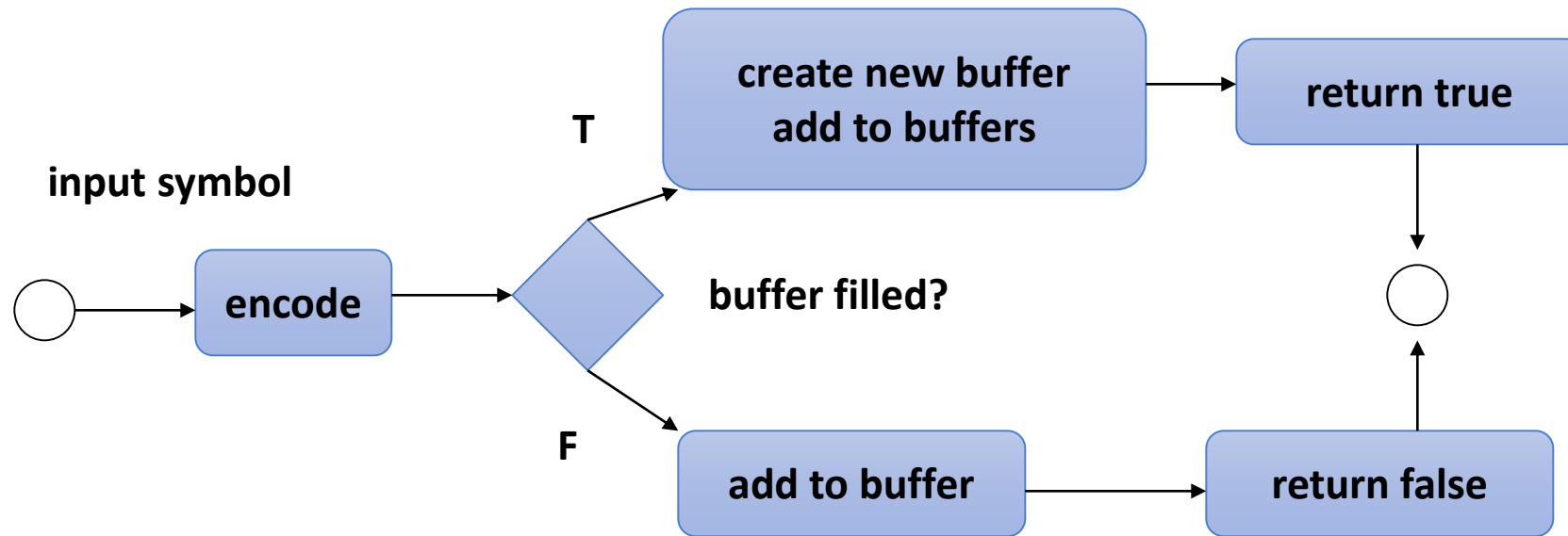
load() puts data in a new symbol table.



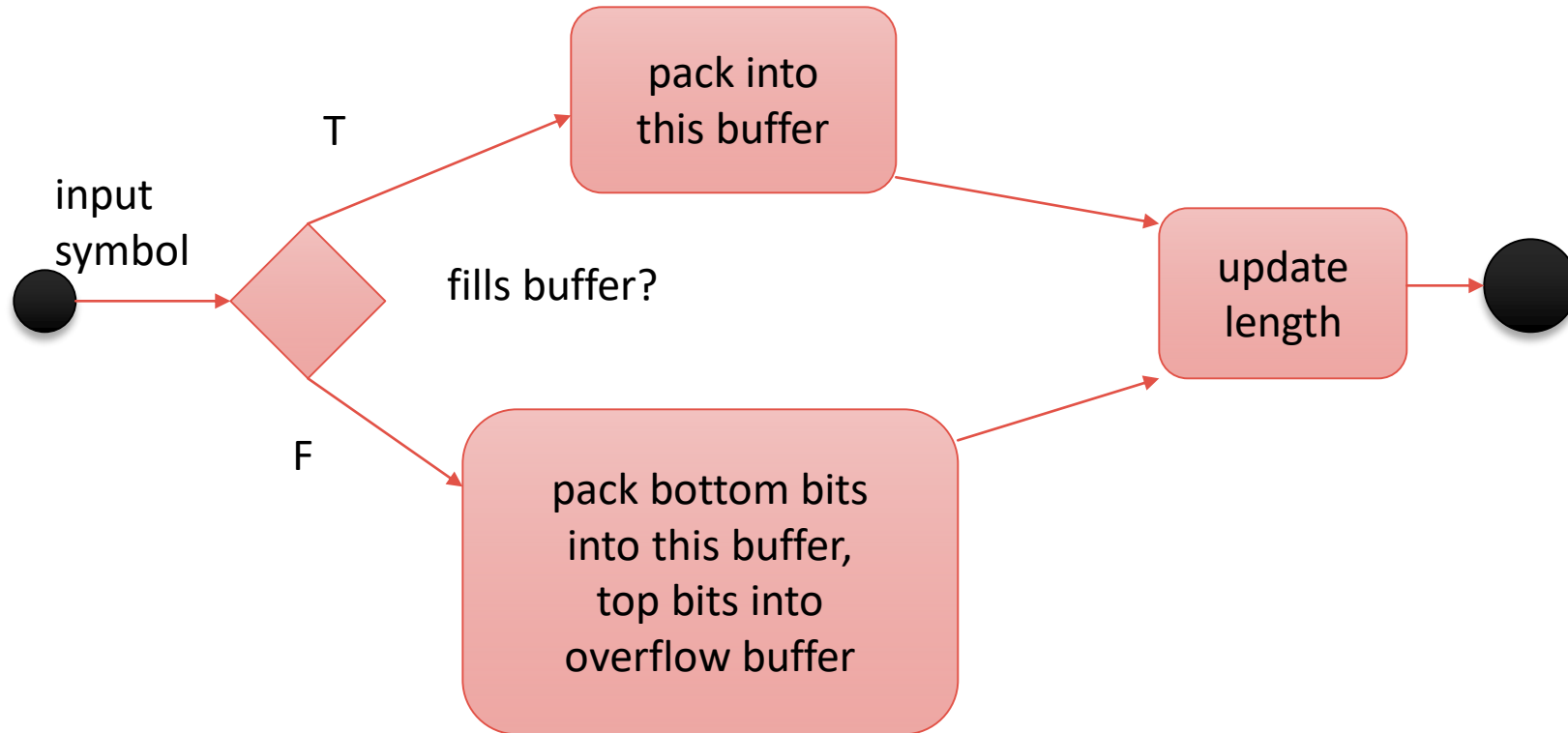
# CLASS RELATIONSHIPS



# ENCODE BEHAVIOR



# INSERT BEHAVIOR



# PROGRAM DESIGN

To build an encoder by using either object-oriented and non-object-oriented implementations

Object-oriented design might be done using C++

Non-object-oriented design might be done using C.

# TESTING

Test by encoding, then decoding:

